Predicting Azerbaijani Property Prices from Scratch: A NumPy Decision Tree and Pegasos SVM on bina.az Listings

Badalov, Sanan Hasanli, Ali Hasanli, Oghuz Sadigov, Sadigov

MLE-AI-201 Machine Learning, AI Academy — Team 07

***Abstract*—**We turn the raw bina.az sale-listings dump (100,775 rows) into a leakage-free modelling set and frame it as two tasks: regression of log(price) (Task A) and classification of a premium/standard price tier split at the training median (Task B). We implement in NumPy a decision tree (Gini, entropy, MSE), a linear soft-margin SVM trained with Pegasos sub-gradient descent, and an RBF-like SVM using random Fourier features, and benchmark them against scikit-learn with matched settings. Hyper-parameters are chosen on a validation split only. On Task A our tree reaches test $R^2 = 0.811$ (RMSE $= 0.340$ in log space), identical to scikit-learn to four decimals. On Task B our tree reaches F1 $= 0.930$, versus 0.793 for the linear SVM and 0.850 for the RFF-SVM; scikit-learn's kernel SVC reaches 0.868. Tree-based models win because price depends on non-linear interactions of area and location. Code: `github.com/BadalovSanan/team07_ml_final_project` (tag `v1.0-final`).

***Index Terms*—**decision tree, support-vector machine, Pegasos, random Fourier features, regression, classification, real-estate price prediction.

I. Introduction

Property prices are a standard tabular-learning problem, and the bina.az dump is realistic: scraped, mixed-type, and full of outliers. Our goals were to (i) clean it and remove target leakage, (ii) implement a decision tree and an SVM *from scratch*, and (iii) compare them honestly with scikit-learn [4]. Contributions: a documented cleaning pipeline; a tested NumPy tree supporting Gini/entropy/MSE and four stopping rules; a Pegasos SVM [1] with margin and support-vector reporting plus a random-Fourier-feature extension [2]; a from-scratch Ridge baseline; and a leakage-free benchmark with cross-validated uncertainty, timing, and error analysis.

II. Background

A tree greedily picks the split maximising impurity decrease [3]: $G = 1 - \sum_k p_k^2$, $H = -\sum_k p_k \log_2 p_k$, and for regression $I_{\text{reg}} = \frac{1}{N}\sum_i (y_i - \bar{y})^2$, with gain $IG = I(\text{parent}) - \sum_c \frac{N_c}{N} I(c)$. The soft-margin SVM minimises the regularised hinge loss, which Pegasos solves by stochastic sub-gradient steps with rate $\eta_t = 1/(\lambda t)$ [1].

III. Data and Preprocessing

Data. The Kaggle "binaaz-sale-project" dump (downloaded 2026-10-14, version 1) has 100,775 rows and 51 columns with Azerbaijani names and text.

Cleaning (all in `src/data_prep.py`). Numeric columns arrive as strings, so we parse area ("145 m^2"; "sot" converted at 1 sot $= 100$ m^2), land area, rooms, and floor ("7 / 9" $\rightarrow$ floor, number of floors). All prices are in AZN, so no conversion is needed. We drop rows with missing price, area, or coordinates; remove content duplicates (same price, area, rooms, floor, coordinates, category, location, and description, since the dump has no exact duplicate ids); and apply *fixed* outlier bounds (not data-derived, hence leak-free): price in $[3\text{k}, 10\text{M}]$ AZN, area in $[10, 50\text{k}]$ m^2, rooms ≤ 20, and coordinates inside Azerbaijan. Structurally missing floor, rooms, and land values (a house has no floor) are filled with 0 plus a missing-indicator, so no statistic is learned from the data. Azerbaijani columns we keep are renamed (`category`, `repair`, `bill_of_sale`, `mortgage`). Categoricals are one-hot encoded over a fixed category list, so every split has identical columns.

Table 1. Dataset summary after cleaning.

Property	Value
Rows raw $\rightarrow$ cleaned	100,775 $\rightarrow$ 75,660
Features used	21
Train / Val / Test split	52,964 / 11,348 / 11,348
Tier threshold (train median)	212,750 AZN
Price-tier balance (std/prem)	50 / 50

Leakage. We drop `unit_price` (price per m^2) and `total_price` (equal to `price` in 99.99% of rows), both arithmetic functions of the target. We also drop identifiers and scrape bookkeeping (ids, URLs, owner and shop names, raw address, timestamps), post-listing popularity and promotion fields (views, vip, featured), free text, currency (constant), and a building-type column with only 154 non-null values.

Tasks and split. Task A predicts log(price) (prices are heavy tailed, Fig. 1); metrics are RMSE, MAE, and R^2, reported in log space and in AZN. Task B labels a listing *premium* if its price exceeds the **training-set median** (212,750 AZN), so the classes are balanced by construction; metrics are precision, recall, F1, ROC-AUC, and the confusion matrix. The 70/15/15 split is stratified on above/below median price, and standardisation statistics come from the training split only (Table 1).

IV. Method

A. Decision tree (from scratch)

`DecisionTree` mirrors the scikit-learn API (`fit`, `predict`, `predict_proba`). For each feature, values are sorted once and all midpoints between distinct neighbours are

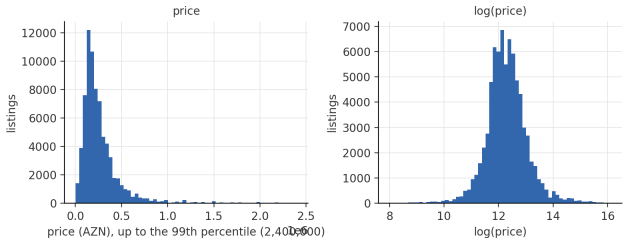


Figure 1. Price (left, clipped at the 99th percentile) is right-skewed; log(price) (right) is close to symmetric, motivating Task A on the log scale.

scored in one vectorised pass using cumulative class counts (classification) or cumulative sums and sums of squares (regression), so the split search is $O(n \log n)$ per feature. The best (feature, threshold) by impurity decrease is taken and the tree recurses. Leaves predict the majority class (with `predict_proba` from leaf frequencies) or the mean. Supported hyper-parameters are `criterion` (Gini, entropy, MSE), `max_depth`, `min_samples_split`, `min_samples_leaf`, and `min_impurity_decrease`; unit tests check that each one changes the tree and that splits match scikit-learn on tiny determined examples and on larger data (`tests/test_decision_tree.py`). Impurity-decrease feature importances are accumulated over all splits.

B. Support-vector machine (from scratch)

We minimise

$$J(\mathbf{w},b) = \tfrac{\lambda}{2}\|\mathbf{w}\|^2 + \tfrac{1}{n}\sum_{i=1}^{n} \max\big(0,\, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)\big) \qquad (1)$$

with labels $y_i \in \{-1,+1\}$. At step t we draw an example; with $\eta_t = 1/(\lambda(t+t_0))$, the update is $\mathbf{w} \leftarrow (1-\eta_t\lambda)\mathbf{w} + \eta_t y_i \mathbf{x}_i [y_i(\mathbf{w}^\top \mathbf{x}_i + b) < 1]$, and $b \leftarrow b + \eta_t y_i$ on margin violations (unregularised). We use $t_0 = 1/\lambda$ so the first steps are ≈ 1 rather than $1/\lambda$, which would make the weights explode for small λ (textbook Pegasos is $t_0 = 0$). Constant and $1/\sqrt{t}$ schedules, mini-batches, and epochs are also exposed. Features are standardised first. After training we report $\|\mathbf{w}\|$, margin width $2/\|\mathbf{w}\|$, and the support vectors $(y_i(\mathbf{w}^\top \mathbf{x}_i + b) \le 1)$.

Non-linearity. We chose random Fourier features (option ii) because they reuse the same linear solver and cost $O(n)$ instead of an $O(n^2)$ kernel matrix: $z(\mathbf{x}) = \sqrt{2/D}\cos(W^\top \mathbf{x} + \mathbf{c}), W \sim \mathcal{N}(0, 2\gamma), c \sim U(0, 2\pi), D = 300$, so that $z(\mathbf{x})^\top z(\mathbf{x}') \approx e^{-\gamma\|\mathbf{x}-\mathbf{x}'\|^2}$. We also wrote a Ridge regression via the normal equations as a Task A baseline.

V. Experimental Setup

Protocol. All seeds are fixed (`SEED=42`) and `python -m src.run_all` regenerates every table and figure. Hyper-parameters are chosen on the *validation* split only: tree depth $\in \{1,\dots,20\}$ and `min_samples_leaf` $\in \{1,\dots,250\}$ (and Gini vs. entropy for Task B); SVM $\lambda \in \{10^{-6},\dots,10^{-1}\}$ and $\gamma \in \{0.048, 0.095, 0.19, 0.38\}$ for RFF; Ridge $\lambda \in \{10^{-3},\dots,10^{3}\}$. The SVM uses 20 epochs, batch size 1. The test set is used once for final numbers. Uncertainty is the mean $\pm$ std of 5-fold cross-validation (stratified for Task B) on train + validation with the chosen settings. Metrics are implemented in NumPy (`src/evaluate.py`).

Baselines (matched). Trees: `DecisionTreeRegressor/Classifier` with the same criterion, depth, and leaf size. Linear SVM: `SGDClassifier` (hinge, L2, `alpha` $= \lambda$, same epochs) in a `Pipeline` with `StandardScaler`. Kernel SVM: `SVC` (RBF, same γ, $C = 1/(\lambda n)$) in a pipeline; because SVC is $O(n^2)$–$O(n^3)$, it is trained on a random subset of at most 10,000 rows. We also include a mean (Task A) and a majority-class (Task B) dummy baseline, and `sklearn.linear_model.Ridge`.

VI. Results

Chosen configurations (validation only). Task A tree: MSE, depth 12, `min_samples_leaf` $= 5$. Task B tree: Gini, depth 20, `min_samples_leaf` $= 1$ (Gini and entropy are within 0.001 val F1: 0.9328 vs. 0.9324). Linear SVM: $\lambda = 10^{-3}$. RFF-SVM: $\lambda = 10^{-5}$, $\gamma = 0.095$. Ridge: $\lambda = 10^{-3}$.

Benchmark. Our tree matches scikit-learn: identical test RMSE (log) and R^2 on Task A, and F1 within 0.0014 on Task B (Tables 2 and 3). The residual differences (MAE in AZN differs by $< 0.2\%$) come from tie-breaking between equal-gain splits and float rounding of midpoint thresholds, not from different algorithms; the unit tests confirm identical splits on determined examples. Our linear SVM matches `SGDClassifier` (F1 0.793 vs. 0.794, AUC 0.868 for both), and our Ridge matches scikit-learn's to all printed digits. Our implementations are slower to fit (tree 3.4× and 8.7× on Tasks A/B; SVM 58×) because the SVM update is a per-example Python loop and scikit-learn uses compiled code. Kernel SVC predicts slowly (5.0 s vs. 0.08 s for RFF) because prediction scales with the number of support vectors, whereas RFF prediction is a fixed-size map plus a dot product.

Tree diagnostics. Fig. 2 shows the bias–variance story. Training error keeps falling with depth while validation error plateaus: on Task B, depth 20 gives train F1 0.996 but val F1 0.933 (depth 10: 0.928/0.914); on Task A, depth 20 gives train RMSE 0.085 but val 0.371, whereas depth 12 reaches val 0.364. Raising `min_samples_leaf` to 5 lowers Task A val RMSE further to 0.353. Learning curves (training-set size 2.6k→53k) show val F1 rising from 0.869 to 0.933 and val RMSE falling from 0.458 to 0.353, still improving, so more data would help. Impurity-decrease importances (Fig. 4) put roughly half the weight on `area_m2` for both tasks, followed by `lat/lng`.

SVM diagnostics. Table 4 shows the trade-off: a larger λ widens the margin (0.062 → 1.853) and shrinks $\|w\|$, while val F1 peaks for $\lambda \in [10^{-4}, 10^{-2}]$ and falls at both extremes (too small: not converged in 20 epochs, $J = 1.79$; too large: underfit). At $\lambda = 10^{-3}$, 27,452 of 52,964 points (52%) are support vectors, i.e. the classes are far from linearly separable. The objective decreases over epochs (Fig. 3): 0.607 → 0.497 for the linear model and 0.516 → 0.378 for RFF, with noise from stochastic updates (RFF spikes at epoch 10, then recovers). RFF improves F1 from 0.793 to 0.850, showing non-linearity matters, and is sensitive to its hyper-parameters (val F1 collapses to 0.53 at γ=0.38, λ=0.1). The linear weights (Fig. 4) push towards premium for `rooms`, `rooms_missing`, and `area_m2`, and towards standard for

Table 2. Task A (regression of log price). Test-set metrics; CV is 5-fold mean ± std on train+val; Fit in seconds on the full training split. AZN columns use $e^{\hat{y}}$.

Model	RMSE (log)	R^2 (log)	MAE (AZN)	RMSE (AZN)	CV RMSE (log)	Fit (s)
Mean baseline	0.7806	0.000	180,660	504,971	0.7789 ± 0.0045	3.11
Ours: tree (MSE)	**0.3397**	**0.811**	78,773	276,905	0.3509 ± 0.0069	1.36
sklearn DecisionTreeRegressor	0.3397	0.811	78,694	276,725	0.3505 ± 0.0065	0.40
Ours: Ridge (λ=10^{-3})	0.5801	0.448	163,061	911,242	0.5849 ± 0.0064	0.008
sklearn Ridge	0.5801	0.448	163,061	911,242	0.5849 ± 0.0064	0.26

Table 3. Task B (price tier, premium = positive). Test-set metrics; CV F1 is 5-fold mean ± std on train+val; Fit/Pred in seconds (Pred on the test set).

Model	Precision	Recall	F1	ROC-AUC	CV F1	Fit (s)	Pred (s)
Majority-class baseline	0.000	0.000	0.000	0.500	0.000 ± 0.000	0.001	0.0001
Ours: tree (Gini)	0.928	0.932	**0.930**	0.932	0.931 ± 0.003	4.82	0.077
sklearn DecisionTreeClassifier	0.930	0.933	0.931	0.934	0.931 ± 0.003	0.56	0.004
Ours: Pegasos SVM (linear)	0.782	0.804	0.793	0.868	0.790 ± 0.004	21.04	0.0003
sklearn SGDClassifier (hinge)	0.782	0.807	0.794	0.868	0.790 ± 0.004	0.36	0.002
Ours: RFF-SVM (RBF approx.)	0.842	0.859	0.850	0.930	0.842 ± 0.005	20.96	0.078
sklearn SVC (RBF, ≤10k rows)	0.864	0.872	0.868	0.949	0.863 ± 0.003	3.29	5.00

Table 4. Linear Pegasos sweep over λ (validation). Margin = $2/\|w\|$; SVs out of 52,964.

λ	$\|w\|$	Margin	#SV	Final J	Val F1
10^{-6}	32.18	0.062	16,727	1.794	0.731
10^{-4}	4.35	0.460	28,604	0.520	0.785
10^{-3}	4.44	0.451	27,452	0.497	**0.786**
10^{-2}	2.53	0.790	30,830	0.554	0.786
10^{-1}	1.08	1.853	37,711	0.654	0.752

land (`cat_Torpaq`), `floor_missing`, and house categories. `rooms` and `rooms_missing` are strongly collinear (missing rooms are filled with 0), so their magnitudes should not be read individually.

Error analysis. Fig. 5 breaks down errors. For Task A the mean absolute log-residual is lowest for apartments (`Yeni tikili` 0.138, `Kohne tikili` 0.135) and highest for land plots (`Torpaq` 0.662, n=544) and commercial objects (`Obyekt` 0.494, n=485); houses sit in between (0.268). The worst test predictions are almost all land plots with areas of 600–36,000 m^2, where price depends on location and use that our features do not capture (e.g. a 14,000 m^2 plot sold for 2.1M AZN was predicted at 49k; a 36,200 m^2 plot at 230k was predicted at 5.0M). For Task B the tree's confusion matrix is 5262/412/384/5290 (TN/FP/FN/TP). Error rate is 13.4% for land and objects but 5.9%–6.1% for apartments, and errors concentrate near the threshold: 19% in the closest quintile to 212,750 AZN versus 2–7% elsewhere. So the tree mostly fails where classes are genuinely ambiguous and in sparse, heterogeneous categories (`Ofis` n=17, `Qaraj` n=8 are too small to judge).

VII. Discussion and Limitations

Why trees beat SVMs here. Price is governed by interactions between area, location, and property type, which axis-aligned threshold splits on `lat`/`lng`/`area` capture directly; a linear boundary cannot (F1 0.793), and Ridge on raw coordinates reaches only $R^2 = 0.448$ (its AZN-scale R^2 is negative because exponentiating log predictions amplifies errors on expensive listings). The RFF-SVM closes part of the gap (0.850) but trails both our tree and scikit-learn's SVC (0.868, trained on only 10k rows). This suggests the shortfall comes from the $D = 300$ approximation and 20 epochs of stochastic optimisation rather than from lack of data.

Limitations. (i) Results use one random split; CV runs on train+val after tuning on val, so CV scores are slightly optimistic, while test numbers are a single hold-out. (ii) The Task B optimum sits at the edge of the depth grid (20), so a deeper grid might help marginally. (iii) SVC uses at most 10k rows for cost reasons, so it is not a like-for-like data budget. (iv) Our SVM is fixed at 20 epochs and is slow because of the per-sample Python loop. (v) Prices are *asking* prices from scraped listings; land, houses, and apartments share one model, and location enters only through coordinates, with no district or time features. (vi) We report ROC-AUC but not PR-AUC.

VIII. Conclusion

A from-scratch NumPy tree reproduces scikit-learn's accuracy on both tasks ($R^2 = 0.811$; F1 = 0.930), and a from-scratch Pegasos SVM reproduces `SGDClassifier` with an inspectable margin and support-vector set. Trees clearly outperform linear and RFF-SVMs on this data, and nearly all errors come from land and commercial listings and from prices near the tier threshold. Next steps would be district-level features, ensembles of our trees, and more RFF components or kernelised Pegasos.

Tools and Acknowledgements

An AI assistant (Claude, Anthropic) was used to draft the text of this report from the repository's result files (`report/*.csv`, `*.md`) and figures; all reported numbers come from those files and from `python -m src.run_all`.

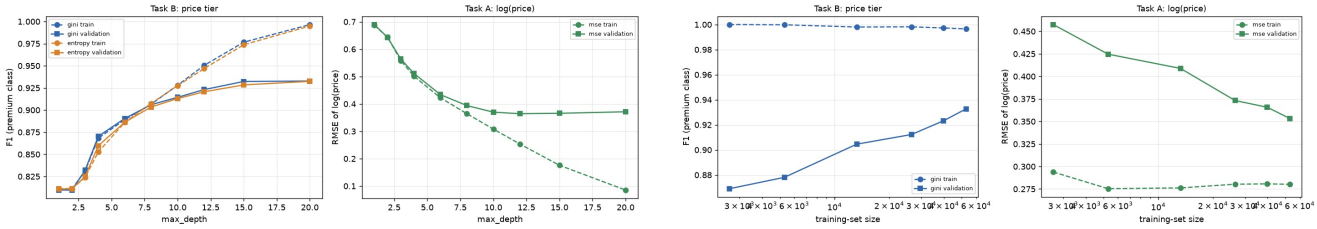


Figure 2. Tree validation curves. Left: F1 (Task B) and RMSE (Task A) vs. `max_depth`; train and validation diverge beyond depth $\approx$10. Right: learning curves vs. training-set size.

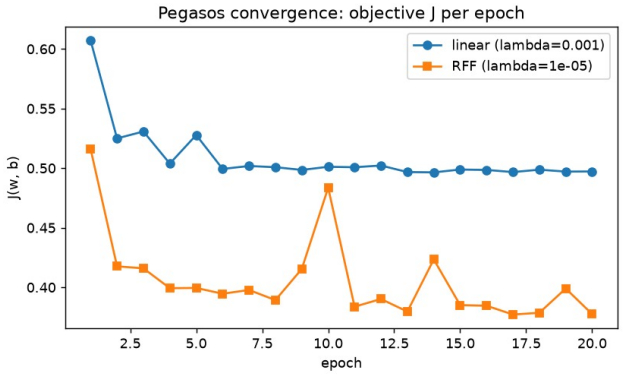


Figure 3. Pegasos objective J per epoch for the linear SVM ($\lambda = 10^{-3}$) and RFF-SVM ($\lambda = 10^{-5}$).

We built on NumPy, pandas, matplotlib, and scikit-learn (baselines only) and on the public Kaggle dataset "binaaz-sale-project" by sehriyarmemmedli [5]. Per-member contributions are listed in `contribution_report.md`.

References

[1] S. Shalev-Shwartz, Y. Singer, N. Srebro, and A. Cotter, "Pegasos: primal estimated sub-gradient solver for SVM," *Mathematical Programming*, vol. 127, no. 1, pp. 3–30, 2011.

[2] A. Rahimi and B. Recht, "Random features for large-scale kernel machines," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2007.

[3] L. Breiman, J. Friedman, R. Olshen, and C. Stone, *Classification and Regression Trees*. Wadsworth, 1984.

[4] F. Pedregosa *et al.*, "Scikit-learn: machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[5] S. Memmedli, "Bina.az Sale Project," Kaggle dataset, 2026. [Online]. Available: `https://www.kaggle.com/datasets/sehriyarmemmedli/binaaz-sale-project`

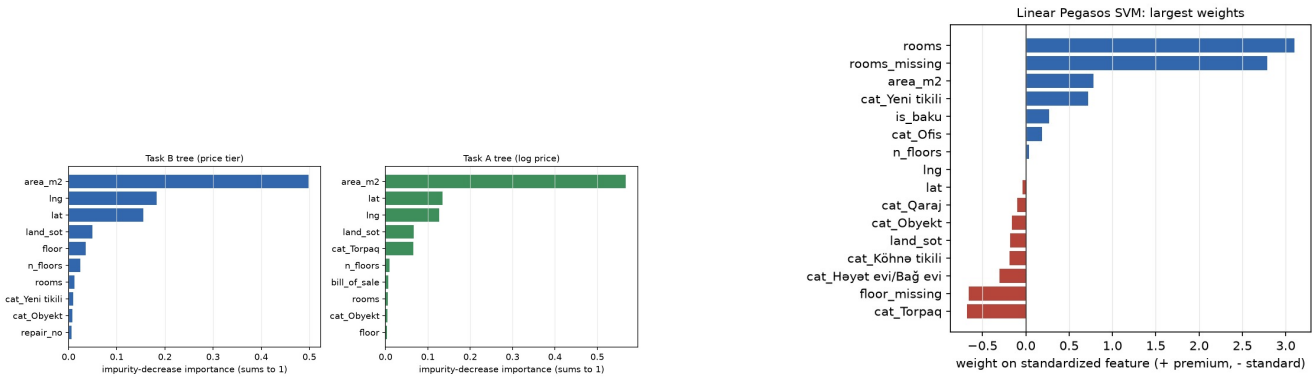


Figure 4. Left: impurity-decrease feature importances of our trees (Task B, Task A). Right: largest weights of the linear Pegasos SVM on standardised features.

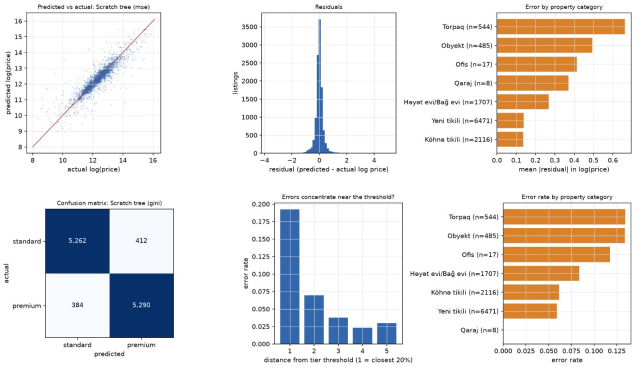


Figure 5. Error analysis of our tree. Top (Task A): predicted vs. actual log price, residuals, and error by category. Bottom (Task B): confusion matrix, error rate by distance to the tier threshold, and error rate by category.